

用 13 个参数学会推理

John X. Morris^{1,2}, Niloofar Mireshghallah^{1,3}, Mark Ibrahim¹, Saeed Mahloujifar¹

¹FAIR at Meta, ²Cornell University, ³Carnegie Mellon University

近期研究表明，语言模型能够学会推理，通常是通过 reinforcement learning 实现的。也有工作尝试为推理训练低秩参数化形式，但传统 LoRA 无法缩放到低于模型宽度的规模。我们质疑：对于学会推理而言，哪怕 rank=1 的 LoRA 是否都仍属必要？为此我们提出 TinyLoRA，这是一种能够将低秩 adapter 缩放到小至单个参数的方法。在这一新参数化下，我们只用 13 个以 bf16 训练的参数（总计 26 字节），就能将 8B 规模的 Qwen2.5 在 GSM8K 上训练到 91% 准确率。我们发现这一趋势具有普遍性：在 AIME、AMC 和 MATH500 等更难的 learning-to-reason benchmark 上，我们用少 1000× 的参数量恢复了 90% 的性能提升。值得注意的是，只有 RL 才能实现如此强的表现；使用 SFT 训练的模型需要大 100-1000× 的更新规模，才能达到同等性能。

Date: May 25, 2026



1 引言

现代语言模型通常会通过 reinforcement learning 的方式，在额外的 thinking tokens 上进行后训练，以获得“推理”能力。此类推理训练通常采用带有可验证奖励的 reinforcement learning (RLVR)，并在数学、代码等高推理密度领域中进行评估 (OpenAI et al., 2024; Shao et al., 2024; Luo et al., 2025)。

尽管很多实践者会为推理任务微调整个模型（通常规模达到数十亿参数）(Guha et al., 2025)，但像 Low-Rank Adaptation (LoRA) (Hu et al., 2021) 这样的参数高效方法，已经把这一需求从数十亿参数降到了数百万“adapter”参数，减少了若干个数量级。即便如此，模型更新仍然很大：对于 Llama3-8B，LoRA 在最小设置 (rank 1) 下也至少需要微调 3M 参数 (Schulman and Lab, 2025)。与一些更早的工作 (Cuccu et al., 2019) 相比，这个参数量仍显得惊人；那些工作表明，在 Atari 这类 toy setting 中，仅靠 6 个神经元就足以解决任务。

用更少参数进行训练有很多好处。微调更少参数可以降低单卡 GPU 的显存占用，也能减少分布式训练中的通信成本。在推理阶段，内存中可以容纳更多 LoRA：adapter 大小缩小 10 倍，就意味着能够并发服务 10 倍数量的 LoRA (Chen et al., 2023)，从而支持大规模高效个性化。已有研究 (Biderman et al., 2024) 还表明，低秩模型更新能够通过保留基座模型中的重要信息来缓解遗忘。

既然如此，我们怎样才能进一步提升参数效率？既有工作主要聚焦于 supervised finetuning (Hu et al., 2021; Li and Liang, 2021; Zaken et al., 2022; Biderman et al., 2024)。我们观察到，在 supervised learning 中想要获得好的性能，需要向模型中吸收远多于 RL 的信息比特。更令人兴奋的是，反过来也成立：当使用 RL 进行微调时，我们可以用小得多的更新规模达到可比的性能。基于这一观察，我们提出一种基于 RL 的替代方案，使得所需微调参数量能够比标准 LoRA 式 SFT 低若干个数量级。

我们考察了传统 LoRA (Hu et al., 2021) 以及更小的 LoRA-XS (Hu et al., 2021)，它们允许我们把更新规模从 10^6 参数缩小到 10^4 参数。在数学任务上，RL 在低参数区间（训练参数少于 100 万）远比 SFT 更有效。在 GSM8K 数学文字题数据集上，Qwen-2.5-7B-Instruct 在只训练 10,000 个参数时，就能从 76% 提升到 95%。由于模型在 10^4 参数下性能退化如此之小，我们进一步提出一种新方法 (TinyLoRA) 来任意缩小 LoRA：其最小更新规模只有 1 个参数。TinyLoRA 是 LoRA 的一种更低秩变体，能够平滑缩放到仅训练单个参数。当我们在 GSM8K 上用 TinyLoRA 和 GRPO 训练 Qwen2.5-7B-Instruct 时，只微调总计 13 个参数，就达到了 91% 的准确率，总更新大小仅为 26 字节。

我们表明，这类超低容量的模型更新只有在 RL 场景下才会成功，而对 SFT 来说，其信息密度不足以支持良好训练效果。在 GSM8K 上，用 GRPO 训练的模型在少于 100 个参数时就能达到 90% 准确率，而相同容量下用 SFT 训练的模型，在训练集上的表现几乎只比基座模型略强。这些发现意味着，RL 所产生的更新在本质上比 SFT 具有更高的信息密度，因此也更适合极小更新规模的训练。

我们还在更复杂的 learning-to-reason benchmark 上评估 TinyLoRA，包括 MATH、AIME 和 AMC。我们再次

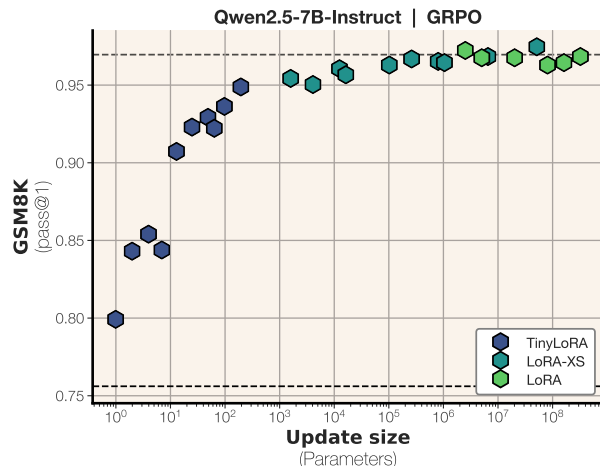


Figure 1 以 Qwen2.5-7B-Instruct 为基座模型时，我们的 TinyLoRA 只用 13 个参数，就能在 GSM8K 上达到距离 full finetuning 仅 5% 以内的性能。虚线表示未训练和 full-FT 基线。

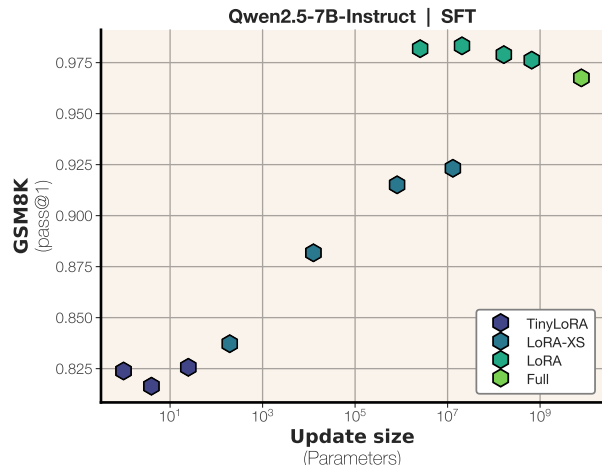


Figure 2 以 Qwen2.5-7B-Instruct 为基座模型时，SFT 只有在更新规模至少达到 1M 参数时效果最好。

观察到它在极低参数区间内仍具备强表现。举一个代表性结果，在 Qwen-2.5-7B-Instruct 上只微调 196 个参数时，跨 6 个高难度数学 benchmark 平均仍能保留其绝对性能提升的 87%。

2 相关工作

参数高效微调。我们将 LoRA 缩放到极低参数数量的进展，是持续改进低秩适配方法这一研究脉络的一部分。相关方法已经大量出现，例如 VeRA (Kopiczko et al., 2023)、VB-LoRA (Li et al., 2024b)、LoRA-XS (Balazy et al., 2025)、UniLoRA (Li et al., 2025)、AdaLoRA (Zhang et al., 2023)、LoRA Drop (Zhou et al., 2024)、NoRA (Lin et al., 2024)、WeightLoRA (Veprikov et al., 2025) 以及 ShareLoRA (Song et al., 2025)。我们并不主张某一种特定初始化或参数化形式，而是聚焦于一些简单方法，尝试把这些技术扩展到极低的参数规模 ($<10K$)；此前绝大多数关于低秩适配的工作都处于 10K 到 10M 参数这一尺度。我们表明，由于我们使用的是更大的模型（数十亿参数）并采用 RL 而非 SFT 训练，因此还存在更高效的参数化可能性。

数据与参数受限条件下的学习。另一条研究线索关注：推理究竟需要多少数据。已有结果发现，对于 SFT 来说，一千个示例就已足够 (Ye et al., 2025)；而在某些设定下，RL 甚至只用一个示例就能学会 (Wang et al., 2025)。我们对更新规模与性能之间 Pareto frontier 的分析，也与一个更一般的原则相关：过参数化模型往往位于一个低内在维度的流形上 (Li et al., 2018; Aghajanyan et al., 2020)。

比较 SFT 与 RL。最近有一系列工作在比较带 reasoning tokens 的 supervised finetuning (SFT) 与 policy gradient 等基于 reinforcement learning 的方法。我们关于数据规模与 LoRA 性能关系的分析，也与 finetuning scaling laws (Zhang et al., 2024a) 以及 LoRA 学习容量研究 (Biderman et al., 2024) 相关。有些工作认为，LoRA 对模型做出的改变只是表层性的 (Shuttleworth et al., 2025)。另一些工作则表明，RL 所产生的更新通常较小且相对稀疏 (Mukherjee et al., 2025)。近期还有多篇论文指出，RL 相比 SFT 具有更好的 out-of-domain 泛化能力 (Chu et al., 2025)、遗忘更少 (Shenfeld et al., 2025)，并且通常是在放大基座模型已有能力，而不是注入全新能力 (Rajani et al., 2025)。同期工作 (Schulman and Lab, 2025) 证明 LoRA 即使在 $r = 1$ 时也能匹配 full finetuning，但并未尝试小于 3M 参数的参数化形式。

3 SFT 与 RL 的更新容量

在本节中，我们重新审视两类流行学习算法，即 supervised finetuning 和 reinforcement learning，并思考用它们把模型训练到良好性能大致需要多少容量。全文中，我们考虑一个模型 π_θ ，它将输入 $x \in \mathcal{X}$ （在本文中即文字题）映射到输出 $y \in \mathcal{Y}$ （解答）。

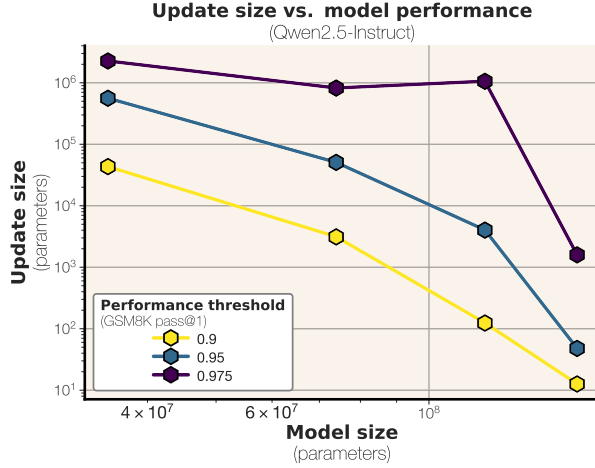


Figure 3 达到最大性能某一阈值所需的最小参数更新规模，与骨干模型大小之间的关系。模型越大，为达到例如 95% 峰值性能所需的更新就越小。

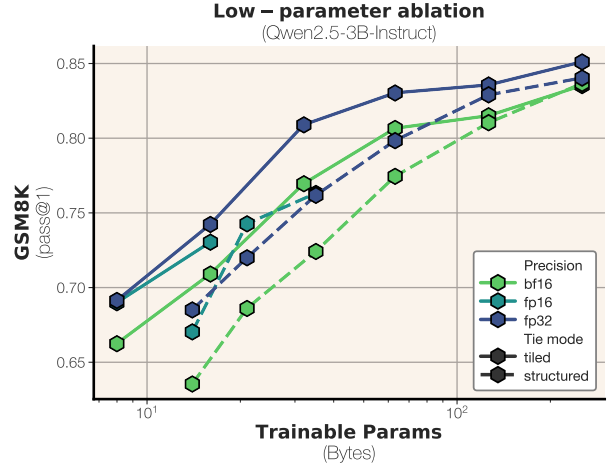


Figure 4 在极小更新规模 (<1KB) 下，使用 Qwen2.5-3B-Instruct 进行的性能消融。令人意外的是，若按 bit-for-bit 计算，使用 fp32 存储参数的表现最好。

我们考虑采用 next-token prediction loss 的 SFT:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(x,y) \sim (\mathcal{X}, \mathcal{Y})} \left[\sum_{t=1}^{|y|} \log \pi_{\theta}(y_t | x, y_{<t}) \right]$$

以及一类称为 policy gradient 的 reinforcement learning 算法:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{x \sim \mathcal{X}; y \sim \pi_{\theta}} \left[\sum_{t=1}^{|y|} \nabla_{\theta} \log \pi_{\theta}(y_t | x, y_{<t}) \cdot R(y) \right]$$

训练数据的信息含量。考虑在样本 (x, y) 上训练模型 θ 。对于模型而言， (x, y) 的信息含量可以看作该样本在模型下的最小描述长度 (Rissanen, 1978)。这一量与 (x, y) 的采样方式无关。

对于 SFT, (x, y) 是从一个期望奖励较高的分布 $(\mathcal{X}, \mathcal{Y})$ 中采样得到的。为简化起见，我们假设奖励是二值的，且底层分布为 $(\mathcal{X}, \mathcal{Y})$ ，其中 $\mathcal{Y} \equiv \pi_{\theta}(\mathcal{X}) \mid r(\mathcal{X}, \pi_{\theta}(\mathcal{X})) = 1$ 。

对于 policy gradient 方法，观测到的数据是一批从 $(\mathcal{X}, (\mathcal{Y}, \mathcal{R})^k)$ 中采样得到的 continuation，其中 $\mathcal{Y} \equiv \pi_{\theta}(\mathcal{X})$ ， $\mathcal{R} \equiv r(\mathcal{X}, \mathcal{Y})$ 。这些样本会在每个 epoch 中基于更新后的模型重新生成。

RL 提供更多数据，但信息更少。从某种意义上说，RL 在训练过程中呈现出的“信息量”看起来比 SFT 更大：在每个 epoch、每个 prompt x 上，我们都会看到 k 个带有新熵值的 continuation，而 SFT 只是跨 epoch 重复使用同一个 continuation。然而，这些信息中的大部分都是噪声。若没有 reward annotation，continuation $\mathcal{Y}$ 本身并不包含提升任务表现所需的有效信号。真正相关的信息完全存在于奖励中，而每个 prompt 所携带的奖励信息只有 $k \cdot H(\mathcal{R})$ 比特；当奖励是二值时，这一上界就是 k 。

信号分离。尽管 RL 中 useful 信息十分稀疏，但信号与噪声却是清晰分离的。与奖励相关的特征会和 r 相关，而与奖励无关的特征则不会。反复重采样会强化这种分离：相关信号会不断累积，而不相关的变化则会相互抵消。

而在 SFT 中，训练信号只是一个不带奖励标注的示例 y 。模型无法区分 y 中哪些特征与任务真正相关。由于缺少将信号与噪声分离开的机制，SFT 只能把所有 token 视作同等重要，因此既要存储有用结构，也要存储无关细节。

假设。我们假设，SFT 在低参数区间表现较差，是因为最小化其目标函数需要模型吸收大量信息比特，而其中只有一部分与任务表现真正相关。相比之下，RL 接收的是更稀疏、更干净的信号，因此能够在更低容量下依然有效学习。

方法	可训练参数	最小值
FT	$\mathcal{O}(nmd^2)$	nmd^2
LoRA	$\mathcal{O}(nmldr)$	$2nmd$
LoRA-XS	$\mathcal{O}(nmr^2)$	nm
VeRA	$\mathcal{O}(nm(d+r))$	$2nmd$
TinyLoRA	$\mathcal{O}(nm u)$	1

Table 1 逐层参数使用量比较，其中每层有 m 个被适配模块，模型宽度为 d ，rank 为 r ，TinyLoRA 的投影维度为 u 。当模块之间采用权重绑定时，TinyLoRA 可缩减到仅一个可训练参数。

4 基于 TinyLoRA 的参数高效微调

reinforcement learning 对容量需求较低，这让我们开始追问：一个 adapter 究竟能小到什么程度，同时仍保持足够的性能？近期工作 (Schulman and Lab, 2025; Yin et al., 2025) 已观察到，LoRA 在更新规模低至 10^6 参数时依然有效。本节在 LoRA 与 LoRA-XS 的基础上提出 TinyLoRA，这是一种能够将参数高效更新进一步缩放到单个训练参数的方法。

4.0.1 LoRA 与 LoRA-XS

低秩适配 (LoRA) (Hu et al., 2021) 通过一个低秩更新来适配被冻结的线性层 $W \in \mathbb{R}^{d \times k}$ ：

$$W' = W + AB$$

其中 $A \in \mathbb{R}^{d \times r}$ 和 $B \in \mathbb{R}^{r \times k}$ 是可训练的，而 W 保持冻结。每个模块的可训练参数量按 $\mathcal{O}(dr)$ 缩放。若在 n 层中的 m 个模块上应用 LoRA，则总参数量为 $\mathcal{O}(nmldr)$ ；对于十亿参数级模型而言，这通常仍是数百万级别 (Biderman et al., 2024; Schulman and Lab, 2025)。LoRA-XS (Balaży et al., 2025) 则把每个模块的参数量从 $\mathcal{O}(dr)$ 降到了 $\mathcal{O}(r^2)$ ：

$$W' = W + U\Sigma RV^\top$$

其中 $U \in \mathbb{R}^{d \times r}$ 、 $\Sigma \in \mathbb{R}^{r \times r}$ 和 $V \in \mathbb{R}^{k \times r}$ 来自对 W 的截断 SVD，而只有 $R \in \mathbb{R}^{r \times r}$ 是可训练的。这可以看作学习如何重组 W 的主奇异方向，并且在实践中优于随机初始化的 LoRA。

4.0.2 用 TinyLoRA 进一步减少可训练参数

由于 LoRA 的规模天然与模型宽度绑定，我们提出一种新技术来进一步缩减训练参数数量。该方法允许我们在模型中仅改变 1 个参数。

缩小 R 的规模。即便在 $r = 1$ 时，LoRA-XS 仍然要求每个被适配模块至少拥有一个参数。我们进一步将其缩小：用一个低维可训练向量 $\mathbf{v} \in \mathbb{R}^u$ 替代 $r \times r$ 矩阵 R ，再通过固定的随机张量 $P \in \mathbb{R}^{u \times r \times r}$ 投影。TinyLoRA 的更新规则为：

$$W' = W + U\Sigma \left(\sum_{i=1}^u v_i P_i \right) V^\top$$

其中 U, Σ, V 来自对 W 的截断 SVD， $P_i \in \mathbb{R}^{r \times r}$ 是固定随机矩阵。每个模块只训练 u 个参数。若在 n 层中的 m 个模块之间进行权重绑定，则总可训练参数量按 $\mathcal{O}(nm u / n_{\text{tie}})$ 缩放；当所有模块共享权重时，它可以进一步缩减到单个参数。

参数共享。既有工作 (Dettners et al., 2023; Biderman et al., 2024) 表明，当 LoRA 同时作用于 MLP 和 attention 模块时效果最好。在 LLaMA-3 (AI@Meta, 2024) 这类典型 transformer 架构中，LoRA 每个 block 会应用 7 次：分别作用于 self-attention 中的 query、key、value、output projection，以及 MLP 中的 up、down 和 gate projection。

对于像 LLaMA-3 70B 这样拥有 80 层的模型，即便在 $u = 1$ （或 LoRA-XS 中 $r = 1$ ）的最小设置下，也仍需要 $80 \times 7 = 560$ 个可训练参数。我们进一步通过在模块之间共享可训练向量 $\mathbf{v}$ 来继续减少参数量。我们将权重绑定因子 n_{tie} 定义为共享同一个 $\mathbf{v}$ 的模块数量，此时对于每层有 m 个模块、共 n 层的模型，总可训练参数量为

$nm u/n_{\text{tie}}$ 。当采用完全权重绑定 ($n_{\text{tie}} = nm$) 时, 所有模块共享一个 $\mathbf{v}$, 总参数量就降为仅 u 个, 最小可至 1 个参数。TinyLoRA 与相关方法的区别见 Table 1。

5 实验

5.1 实验设计

我们的实验目标是在尽量减少训练参数数量的前提下, 提升语言模型的数学推理能力。我们在 GSM8K (Cobbe et al., 2021) (一个包含 7,500 道数学文字题的数据集) 以及更难的 MATH 训练集 (Hendrycks et al., 2021) 上进行评估。

我们考虑两类训练方式: *supervised finetuning* (SFT, 即 next-token prediction), 以及 *reinforcement learning*, 具体采用 Group Relative Policy Optimization (GRPO) (Shao et al., 2024)。实验对象是来自 Llama-3 (AI@Meta, 2024) 与 Qwen-2.5 (Qwen et al., 2025) 家族的 instruction-tuned 语言模型。

我们设置了四个基线: full finetuning、LoRA、LoRA-XS 和 TinyLoRA。对于所有 LoRA 变体, 我们测试 ranks $\{1, 8, 64, 256\}$; 对于 TinyLoRA, 我们考虑共享 $\{1, 8, 64, 256\}$ 层。所有 RL 实验均使用 exact-match reward。

对于 GSM8K, 我们不使用任何 KL penalty, 并在完整数据集上训练 3 个 epoch, 每道题采样 4 个响应, batch size 为 64。采样时最大生成长度设为 4096。

对于 MATH 训练, 我们遵循 SimpleRL (Zeng et al., 2025) 的设置, 它使用了一个更大的训练数据集, 其中包含 GSM8K 和 MATH 等子集。数据按照难度分层, 我们只在最难那一档上训练, 该部分共有 8,523 道题。我们使用最大 prompt 长度 1024、最大 response 长度 3072 token 生成数据, 并采用 ‘boxed’ chat template。其余超参数与 SimpleRL 保持一致: KL coefficient 为 0.001, temperature 为 1.0, batch size 为 256, 每个 response 生成 8 个样本。

由于更新规模的变化已知会影响有效学习率 (Biderman et al., 2024; Schulman and Lab, 2025), 我们在学习率 $\{10^{-7}, 5 \times 10^{-7}, 10^{-6}, 5 \times 10^{-6}, 10^{-5}, 10^{-4}, 2 \times 10^{-4}\}$ 上进行 sweep, 并在每个更新规模下选取表现最好的学习率, 结果取 3 个随机种子的平均值。

评估。对于仅在 GSM8K 上训练的模型, 我们在 GSM8K 验证集上评估。对于 MATH 实验, 我们还额外评估了 SimpleRL (Zeng et al., 2025) 所采用的标准数据集: MATH500 (Hendrycks et al., 2021)、Minerva (Lewkowycz et al., 2022)、GAOKAO (Zhang et al., 2024b)、OlympiadBench (He et al., 2024)、CollegeMath (Feynman Innovations, 2024)、AIME 24 (Li et al., 2024a) 和 AMC23 (He et al., 2024)。这些数据集都属于不同难度层次的数学文字题, 并来自标准的 Qwen-Math evaluation (Qwen et al., 2025)。

在 vLLM 中实现 TinyLoRA。我们所有 RL 实验都运行在开源 VERL 框架 (Sheng et al., 2024) 中, 并使用 vLLM (Kwon et al., 2023) 做推理。然而, vLLM 需要为 LoRA 编写自定义 kernel, 且只支持最小 rank 为 4 的 LoRA (也不支持 LoRA 变体)。我们通过修改训练流程来绕开这一问题: 推理时使用合并后的模型权重, 而只有最后一次 forward pass 使用真正的 LoRA 模型。这会自然带来训练与推理之间的数值不匹配, 我们使用 truncated importance sampling (Ionides, 2008; Yao et al., 2025) 来缓解这一问题。这样一来, 我们就能够在不为每种新方法分别实现底层代码的情况下测试新的参数高效方法。

6 结果

6.1 初始低参数结果: TinyLoRA、LoRA-XS 与 LoRA

我们首先在 GSM8K 上使用 GRPO 训练并评估 Qwen2.5-7B-Instruct, 结果如 Figure 1 所示。我们观察到, 从 TinyLoRA (训练 $1 - 10^3$ 个参数) 到 LoRA-XS (训练 $10^3 - 10^6$ 个参数) 再到 LoRA (训练 $> 10^6$ 个参数) 之间, 性能变化呈现出相当平滑的过渡。只训练 120 个参数, 我们就能恢复 GSM8K 上 95% 的净性能提升。更进一步, 使用 TinyLoRA 时, 只训练单个参数就能带来 4% 的性能提升。

参数量	GSM8K	MATH 500	Minerva Math	Olympiad Bench	AIME24	AMC23	平均
Qwen2.5-3B-Instruct							
(0)	76.0	55.0	18.5	21.3	2.1	23.4	32.7
16	80.9	64.0	19.9	23.0	3.0	31.5	37.1
63	85.3	64.1	20.1	26.6	7.3	36.0	39.9
252	85.4	66.4	28.3	29.3	13.3	47.5	45.0
504	86.1	66.6	28.7	30.8	16.7	47.5	46.1
8,064	87.2	67.8	28.3	30.7	10.0	47.5	45.2
129,024	86.7	67.8	29.4	32.3	10.0	55.0	46.9
(3,085,846,528)	87.0	69.0	31.7	33.1	15.0	52.2	48.0
Qwen2.5-7B-Instruct							
(0)	88.2	64.6	25.7	30.1	3.3	30.0	40.3
13	91.8	74.6	27.1	36.3	16.0	54.5	50.1
49	91.5	74.2	26.6	37.2	12.6	55.5	49.6
196	92.2	76.6	37.1	38.8	16.7	57.5	53.2
392	92.2	77.0	35.7	40.1	16.7	65.0	54.4
6,272	91.9	78.0	37.5	41.0	16.7	57.5	53.8
100,352	92.8	78.0	37.1	43.3	16.7	60.0	54.6
(7,615,487,488)	91.7	78.2	38.6	40.4	20.0	62.5	55.2
Qwen2.5-7B-Math							
(0)	65.5	63.6	12.5	25.8	13.3	42.5	37.2
196	72.5	62.0	26.5	31.4	20.0	50.0	43.7
392	86.0	74.8	31.6	37.9	26.7	60.0	52.8
6,272	87.0	77.4	28.7	40.0	16.7	67.5	52.9
100,352	87.0	78.6	32.7	39.0	30.0	62.5	55.0
(7,615,487,488)	90.2	80.2	37.5	39.0	40.0	70.0	59.5

Table 2 Qwen2.5 系列模型在数学推理任务上的表现。(0) 表示未训练的基座模型；括号中的大数值表示 full finetuning (无 LoRA)。所有实验均使用 GRPO 算法并训练 3 个 epoch。full finetuning 与基线结果引自 (Zeng et al., 2025)。

6.2 SFT 对比 RL

我们展示了 RL (Figure 1) 与 SFT (Figure 2) 的完整 sweep 结果。结果发现，在低参数量区域，RL 的效率显著更高：基线为 76% 时，RL 训练的模型在 13 个参数下可达到 91% (绝对提升 15%)，在 120 个参数下可达 95%。而 SFT 训练的模型则明显更差：13 个参数时只有 83%，120 个参数时只有 84%。

我们还观察到，从高 rank 的 LoRA-XS 过渡到低 rank 的 LoRA 时，变化没有那么平滑。我们注意到，直接在答案上做 SFT 训练本质上是一种 *off-policy* 学习，而当训练对象直接变成模型输出时，其动态过程可能会发生变化 (Mukherjee et al., 2025; Chen et al., 2025)。

6.3 在 MATH 上训练

我们研究了低参数适配方法在 SimpleRL 训练框架 (Zeng et al., 2025) 下的表现。我们针对 7B 的 Qwen 模型，在不同更新规模下进行训练，并在数学推理任务上评估，结果见 Table 2。我们考察的更新规模为 {100, 10,000, 1M} 参数，并与 full finetuning 进行对比。

我们还将训练过程中的指标随步骤变化进行了可视化 (Figure 5)，包括平均奖励与平均响应长度。更大的参数量通常能获得更高奖励，并生成更长响应；但所有参数规模都会获得一定奖励，哪怕更新规模小到只有 16 个参数。进一步可视化训练与推理阶段 log probability 之间的 KL divergence 后，我们发现训练模型与推理模型之间的 KL 几乎可以忽略，这表明我们在每个训练步骤合并 LoRA 权重的技术是有效的。

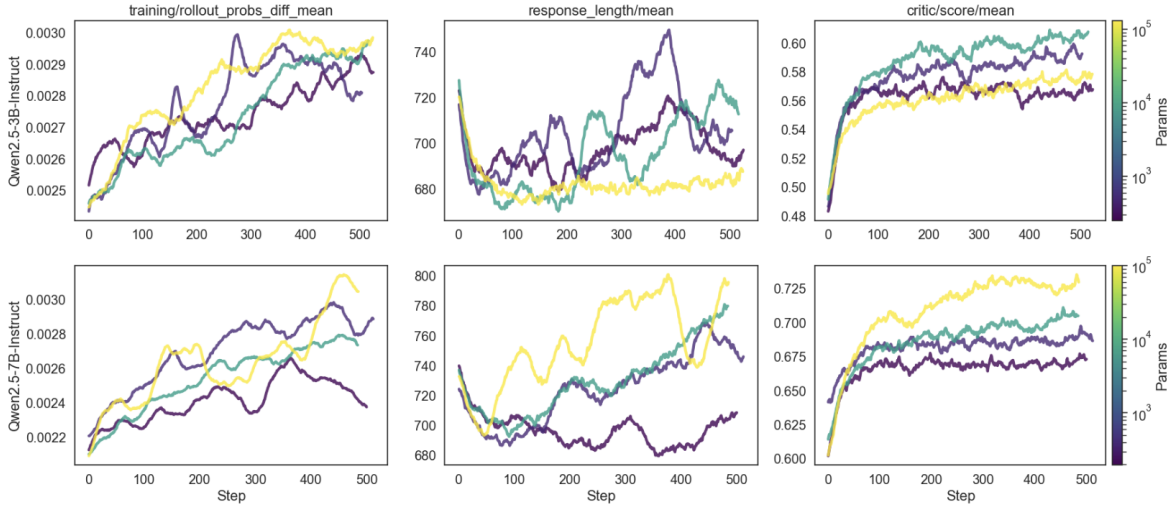


Figure 5 TinyLoRA 在 MATH 上训练过程中的表现。

6.4 跨模型骨干的扩展

接着，我们将 TinyLoRA 从 XS 的 $r = 1$ 情形 (np 个参数) 一路缩放到只训练单个参数，并在 GSM8K 上对 Qwen3-8B 进行测试。结果如 Figure 1 所示。对于 GSM8K 这一任务，在所有规模上，Qwen 的参数效率都高于 LLAMA。从绝对表现看，Qwen 的效率相当惊人：只训练 13 个参数就能达到 94.7% 准确率。即便只训练 1 个参数，Qwen 仍能获得接近 82% 的准确率（相较基线约提升 5%）。这一发现或许佐证了近期一些观察：Qwen 在预训练过程中可能见过相似样例 (Wu et al., 2025)。

即便在允许单参数适配单层的 LoRA-XS 情况下，我们仍然受限于模型中线性层的总数。大模型中可能包含数百个线性层，这意味着在 $r = 1$ 时仍然需要数百字节的存储。经验上我们观察到，Qwen 只用几百个参数依然能几乎保持同样好的学习效果。

尽管 LLAMA 在 full finetuning 下具有相似的上界，但在这一低参数设定中，LLAMA 对小更新规模的响应不如 Qwen：它在 1KB 的更新规模（以 bf16 训练 500 个参数）下可达到 85%。与 Qwen 不同，当训练参数少于 5 个时，LLAMA 几乎无法在基线上带来明显提升。

从 rank 1 到 128。我们在 Figure 1 中可视化了 Qwen 与 LLAMA 从 $r = 1$ 到 $r = 128$ 的性能变化。这对应的更新规模从 1KB ($r = 1$) 一直延伸到 8MB ($r = 128$)。我们看到性能呈现出稳定的单调上升趋势，这表明虽然单位参数带来的收益在递减，但继续微调更多参数在大约 2MB 以内仍然是有帮助的。

从 LoRA-XS 到 LoRA，跨骨干模型比较。我们在 Figure 6 中展示了不同骨干模型规模下 LoRA 和 LoRA-XS 的结果。结论是混合的：在最小模型上，LoRA-XS 明显优于 LoRA，这与 (Balazy et al., 2025) 的发现一致。但随着模型规模增大，这一趋势逐渐减弱，并退化为主要由训练参数数量决定的性能缩放。（需要注意的是，更大模型中的 LoRA 模块数量显著增加，这可能正是关键差异所在。）

总体而言，低参数适配在更大的模型上效果更好；在这种情况下，更小的 adapter 似乎更接近性能上限。我们在 Figure 3 中绘制了 Qwen2.5-Instruct 家族不同模型上的更新规模与相对性能提升之间的关系。趋势非常清晰：随着模型规模增大，只需更少的绝对参数，就能把模型编程到接近其 full finetuning 性能的 95%。这些趋势说明，极大规模（如万亿级）模型未来可能只需极少数参数就能在许多任务上完成训练。

6.5 比特受限场景下的扩展

我们进一步追问：当参数量已经很小（几十到几百个参数），且限制变成总更新大小（以字节计）时，TinyLoRA 应该如何继续扩展？这种场景在分布式训练中很常见，因为模型参数更新的通信本身可能就是主要瓶颈。我们考虑两种参数共享策略：structured，即相邻且同类型模块共享参数；以及 tiled，即相邻且深度相近的模块共享参数，而不关心具体类型。

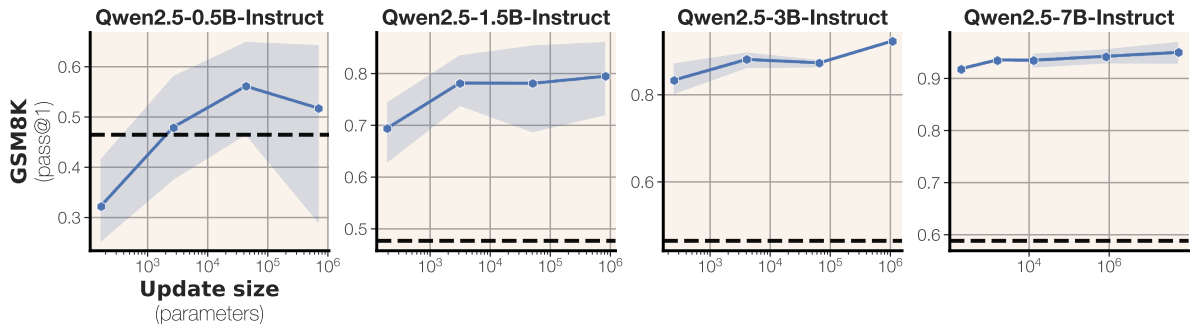


Figure 6 TinyLoRA 在不同骨干规模下的表现 (GSM8K GRPO)，虚线表示未训练基线。小更新只会显著改善较大的模型。

Figure 4 展示了不同共享策略和不同精度下的结果。令人意外的是，tiling 优于 structured sharing；例如在 query projection 模块之间共享参数，并没有带来收益。在所有层都共享、且使用 float16 精度时，Qwen 仍然能在 GSM8K 上训练到 70% 左右，相比基线绝对提升超过 10%。我们还注意到，即便考虑到 fp32 在字节上要大一倍，它的表现仍优于 bf16 和 float16。

7 消融实验

我们训练了一批较小模型，以分析 TinyLoRA 超参数的影响。使用 GRPO，我们对 Qwen-2.5-3B-Instruct 和 LLaMA-3.2-3B-Instruct 进行微调，并在冻结的 SVD rank r 、可训练投影维度 u ，以及权重绑定因子 n_{tie} （即共享同一个 $\mathbf{v}$ 的模块数量）上进行 sweep。

冻结 rank r 的选择。我们首先分析性能随冻结 rank r 变化的情况 (Figure 7)。虽然人们可能会直觉地认为保留更多奇异方向会提高表达能力，但我们发现回报在递减：把 r 从 1 提升到 2 只能带来有限增益，而更大的值反而会降低性能。我们猜测，这是因为更高的 rank 会在冻结的 U 、 Σ 、 V 组件中引入更多自由度，使得小型可训练向量 $\mathbf{v}$ 更难优化。基于这些结果，我们在所有主要实验中都采用 $r = 2$ 。

u 与 n_{tie} 的权衡。在固定参数预算下，TinyLoRA 需要在每个模块的投影维度 u 与权重绑定因子 n_{tie} 之间做平衡。增大 u 会提升每个模块的表达能力，但也会增加参数量；增大 n_{tie} 能减少参数，却会迫使多个模块共享同一个 $\mathbf{v}$ 。我们在 Figure 8 中绘制了这种权衡关系。总体来看，性能会随着更大的 u 和更小的 n_{tie} 而提升，也就是说，单模块更新越有表达力、共享越少，效果越好。这给实践者一个清晰经验：在继续增大 n_{tie} 之前，应先尽可能把 u 的预算用满（最低也要保证 $u = 1$ ）。

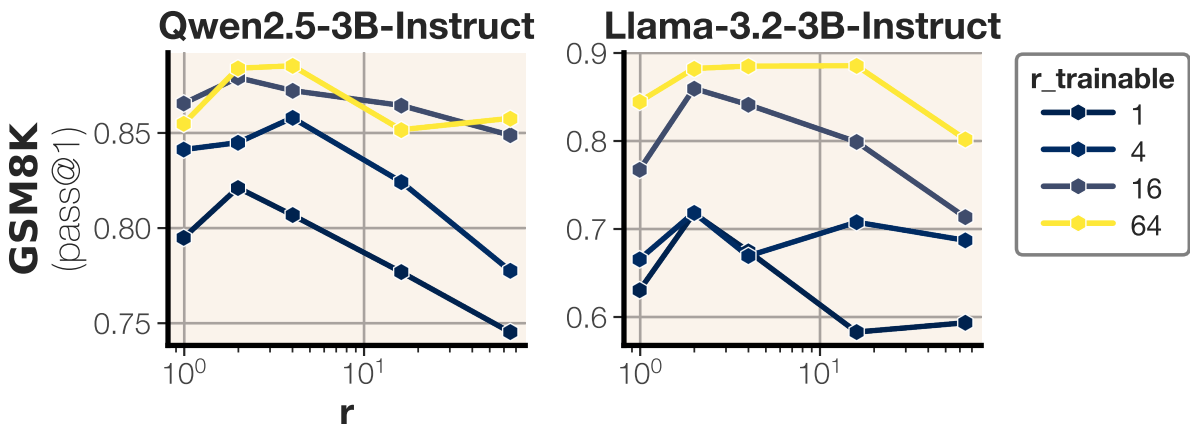


Figure 7 TinyLoRA 在不同冻结 rank r 与可训练 rank $r_{trainable}$ 下的表现。

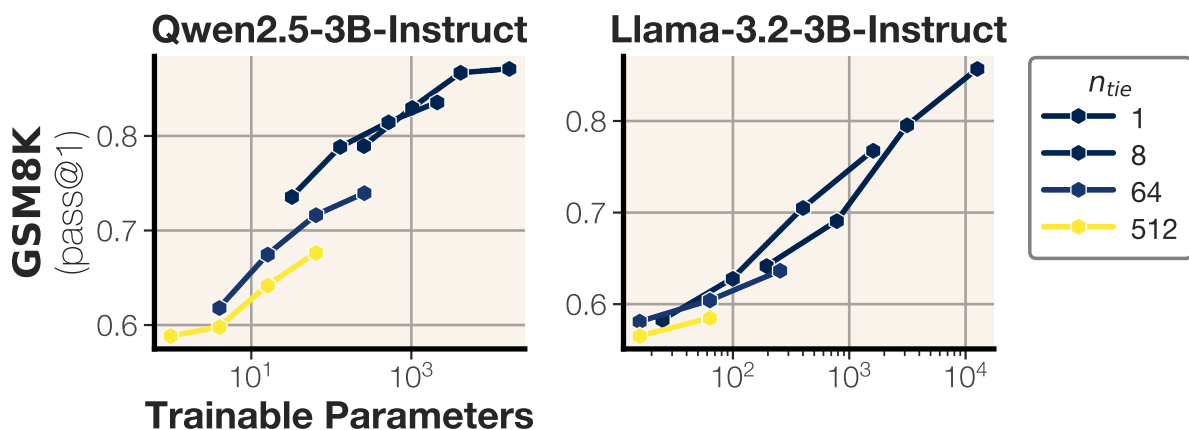


Figure 8 Qwen2.5-3B-Instruct 在不同绑定层数与可训练 rank 下的性能消融。

8 讨论

我们的工作提出了一些有趣问题：微调期间模型更新规模，究竟如何随着模型规模以及学习目标而变化？

缩放趋势。我们的发现表明，随着模型规模增大，LoRA 在更小参数量下会变得更有效。在固定数据集条件下，更大的模型可以用更少参数加以控制。这一趋势意味着，随着模型继续扩展，像 TinyLoRA 这样极端参数高效的方法只会越来越受欢迎。

这怎么可能？人们可能会问，为什么像 GSM8K 这样困难的任务，竟然能只靠 13 个参数学会求解？一种解释是，完成任务所需的知识其实早已存储在模型参数之中，而真正需要改变的只是完成任务所需的风格。尤其是，学会生成更长输出这件事，可能只需极少参数即可实现，而这已被证明能够显著提升数学和推理任务表现 (Shao et al., 2024)。

Qwen 与 LLaMA 的特殊性。我们还观察到，在小更新规模下，Qwen-2.5 模型整体上明显优于 LLaMA-3。通常来说，同一家族中的 Qwen 模型只需大约少 10 倍的更新参数，就能达到与 LLaMA 相当的性能。这可能与模型架构细节或预训练过程中的差异有关。

数学数据集的局限。我们需要强调，本文发现目前仍局限于数学数据集。很多结果似乎都在表明：使用 reinforcement learning 时，极小更新（只有数百字节量级）就足以支持模型以合理性能学会推理。但这些结论仍然局限于数学式推理的范围，是否能推广到科学、创意写作等其他领域，尚未可知。我们选择聚焦数学，是因为截至 2025 年，数学仍是 reasoning model 最受欢迎的应用场景；不过我们也期待这些发现未来能在其他领域被扩展和复现。

9 结论

我们提出了 TinyLoRA，并表明，有效模型所需的调优参数量其实可以远低于以往的认知。TinyLoRA 所产生的更新，往往能以极小文件体积逼近 full finetuning 的性能，很多情况下总大小甚至不到 1KB。我们的结果说明，一些模型在只吸收极少数字节信息的前提下，就能学会完成某些任务，这也促使我们重新思考：带验证的 RL 过程中，模型究竟学到了什么。最后，我们还表明，这种现象只会在大模型上的 RL 场景中出现；如果不用 RL，或模型规模不够大，则效果不会如此显著。

10 致谢

感谢 Jessy Lin、Dan Biderman、Sasha Rush、John Schulman 和 Kamalika Chaudhuri 为本工作提供的宝贵反馈。

References

- Armen Aghajanyan, Luke Zettlemoyer, and Sonal Gupta. Intrinsic dimensionality explains the effectiveness of language model fine-tuning, 2020. URL <https://arxiv.org/abs/2012.13255>.
- AI@Meta. Llama 3 model card. 2024. URL https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md.
- Klaudia Bałazy, Mohammadreza Banaei, Karl Aberer, and Jacek Tabor. Lora-xs: Low-rank adaptation with extremely small number of parameters, 2025. URL <https://arxiv.org/abs/2405.17604>.
- Dan Biderman, Jacob Portes, Jose Javier Gonzalez Ortiz, Mansheej Paul, Philip Greengard, Connor Jennings, Daniel King, Sam Havens, Vitaliy Chiley, Jonathan Frankle, Cody Blakeney, and John P. Cunningham. Lora learns less and forgets less, 2024. URL <https://arxiv.org/abs/2405.09673>.
- Howard Chen, Noam Razin, Karthik Narasimhan, and Danqi Chen. Retaining by doing: The role of on-policy data in mitigating forgetting, 2025. URL <https://arxiv.org/abs/2510.18874>.
- Lequn Chen, Zihao Ye, Yongji Wu, Danyang Zhuo, Luis Ceze, and Arvind Krishnamurthy. Punica: Multi-tenant lora serving, 2023. URL <https://arxiv.org/abs/2310.18547>.
- Tianzhe Chu, Yuexiang Zhai, Jihan Yang, Shengbang Tong, Saining Xie, Dale Schuurmans, Quoc V. Le, Sergey Levine, and Yi Ma. Sft memorizes, rl generalizes: A comparative study of foundation model post-training, 2025. URL <https://arxiv.org/abs/2501.17161>.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>.
- Giuseppe Cuccu, Julian Togelius, and Philippe Cudre-Mauroux. Playing atari with six neurons, 2019. URL <https://arxiv.org/abs/1806.01363>.
- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms, 2023. URL <https://arxiv.org/abs/2305.14314>.
- Feynman Innovations. Maths-college. <https://huggingface.co/datasets/ajibawa-2023/Maths-College>, 2024. revision 9eaa32a.
- Etash Guha, Ryan Marten, Sedrick Keh, Negin Raoof, Georgios Smyrnis, Hritik Bansal, Marianna Nezhurina, Jean Mercat, Trung Vu, Zayne Sprague, Ashima Suvarna, Benjamin Feuer, Liangyu Chen, Zaid Khan, Eric Frankel, Sachin Grover, Caroline Choi, Niklas Muennighoff, Shiye Su, Wanxia Zhao, John Yang, Shreyas Pimpalgaonkar, Kartik Sharma, Charlie Cheng-Jie Ji, Yichuan Deng, Sarah Pratt, Vivek Ramanujan, Jon Saad-Falcon, Jeffrey Li, Achal Dave, Alon Albalak, Kushal Arora, Blake Wulfe, Chinmay Hegde, Greg Durrett, Sewoong Oh, Mohit Bansal, Saadia Gabriel, Aditya Grover, Kai-Wei Chang, Vaishaal Shankar, Aaron Gokaslan, Mike A. Merrill, Tatsunori Hashimoto, Yejin Choi, Jenia Jitsev, Reinhard Heckel, Maheswaran Sathiamoorthy, Alexandros G. Dimakis, and Ludwig Schmidt. Openthoughts: Data recipes for reasoning models, 2025. URL <https://arxiv.org/abs/2506.04178>.
- Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Leng Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, Jie Liu, Lei Qi, Zhiyuan Liu, and Maosong Sun. Olympiadbench: A challenging benchmark for promoting agi with olympiad-level bilingual multimodal scientific problems, 2024. URL <https://arxiv.org/abs/2402.14008>.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset, 2021. URL <https://arxiv.org/abs/2103.03874>.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021. URL <https://arxiv.org/abs/2106.09685>.
- Edward L. Ionides. Truncated importance sampling. *Journal of Computational and Graphical Statistics*, 17(2):295–311, 2008. doi: 10.1198/106186008X320456. URL <https://doi.org/10.1198/106186008X320456>.
- Dawid J. Kopiczko, Tijmen Blankevoort, and Yuki M. Asano. Vera: Vector-based random matrix adaptation. *arXiv preprint arXiv:2310.11454*, 2023. URL <https://arxiv.org/abs/2310.11454>. Revised version v2, 16 Jan 2024.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention, 2023. URL <https://arxiv.org/abs/2309.06180>.

- Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, Yuhuai Wu, Behnam Neyshabur, Guy Gur-Ari, and Vedant Misra. Solving quantitative reasoning problems with language models, 2022. URL <https://arxiv.org/abs/2206.14858>.
- Chunyu Li, Heerad Farkhor, Rosanne Liu, and Jason Yosinski. Measuring the intrinsic dimension of objective landscapes, 2018. URL <https://arxiv.org/abs/1804.08838>.
- Jia Li, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Costa Huang, Kashif Rasul, Longhui Yu, Albert Jiang, Ziju Shen, Zihan Qin, Bin Dong, Li Zhou, Yann Fleureau, Guillaume Lample, and Stanislas Polu. NuminaMath, 2024a. URL https://github.com/project-numina/aimo-progress-prize/blob/main/report/numina_dataset.pdf.
- Kaiyang Li, Shaobo Han, Qing Su, Wei Li, Zhipeng Cai, and Shihao Ji. Uni-lora: One vector is all you need. *arXiv preprint arXiv:2506.00799*, 2025. URL <https://arxiv.org/abs/2506.00799>.
- Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation, 2021. URL <https://arxiv.org/abs/2101.00190>.
- Yang Li, Shaobo Han, and Shihao Ji. Vb-lora: Extreme parameter efficient fine-tuning with vector banks, 2024b. URL <https://arxiv.org/abs/2405.15179>.
- Cheng Lin, Lujun Li, Dezhi Li, Jie Zou, Wei Xue, and Yike Guo. Nora: Nested low-rank adaptation for efficient fine-tuning large models, 2024. URL <https://arxiv.org/abs/2408.10280>.
- Michael Luo, Sijun Tan, Justin Wong, Xiaoxiang Shi, William Y. Tang, Manan Roongta, Colin Cai, Jeffrey Luo, Li Erran Li, Raluca Ada Popa, and Ion Stoica. DeepScaler: Surpassing o1-preview with a 1.5b model by scaling rl. <https://pretty-radio-b75.notion.site/DeepScaleR-Surpassing-O1-Preview-with-a-1-5B-Model-by-Scaling-RL-19681902c1468005bed8ca303013a4e2>, 2025. Notion Blog.
- Sagnik Mukherjee, Lifan Yuan, Dilek Hakkani-Tur, and Hao Peng. Reinforcement learning finetunes small subnetworks in large language models, 2025. URL <https://arxiv.org/abs/2505.11711>.
- OpenAI, :, Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry ..., Yuchen He, Yuchen Zhang, Yunyun Wang, Zheng Shao, and Zhuohan Li. Openai o1 system card, 2024. URL <https://arxiv.org/abs/2412.16720>.
- Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report, 2025. URL <https://arxiv.org/abs/2412.15115>.
- Neel Rajani, Aryo Pradipta Gema, Seraphina Goldfarb-Tarrant, and Ivan Titov. Scalpel vs. hammer: Grpo amplifies existing capabilities, sft replaces them, 2025. URL <https://arxiv.org/abs/2507.10616>.
- Jorma Rissanen. Modeling by shortest data description. *Automatica*, 14(5):465–471, 1978.
- John Schulman and Thinking Machines Lab. Lora without regret. *Thinking Machines Lab: Connectionism*, 2025. doi: 10.64434/tml.20250929. <https://thinkingmachines.ai/blog/lora/>.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL <https://arxiv.org/abs/2402.03300>.
- Idan Shenfeld, Jyothish Pari, and Pulkit Agrawal. RL’s razor: Why online reinforcement learning forgets less, 2025. URL <https://arxiv.org/abs/2509.04259>.
- Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv: 2409.19256*, 2024.
- Reece Shuttlesworth, Jacob Andreas, Antonio Torralba, and Pratyusha Sharma. Lora vs full fine-tuning: An illusion of equivalence, 2025. URL <https://arxiv.org/abs/2410.21228>.
- Yurun Song, Junchen Zhao, Ian G. Harris, and Sangeetha Abdu Jyothi. Sharelora: Parameter efficient and robust large language model fine-tuning via shared low-rank adaptation, 2025. URL <https://arxiv.org/abs/2406.10785>.

- Andrey Veprikov, Vladimir Solodkin, Alexander Zyl, Andrey Savchenko, and Aleksandr Beznosikov. Weightlora: Keep only necessary adapters, 2025. URL <https://arxiv.org/abs/2506.02724>.
- Yiping Wang, Qing Yang, Zhiyuan Zeng, Liliang Ren, Liyuan Liu, Baolin Peng, Hao Cheng, Xuehai He, Kuan Wang, Jianfeng Gao, Weizhu Chen, Shuohang Wang, Simon Shaolei Du, and Yelong Shen. Reinforcement learning for reasoning in large language models with one training example, 2025. URL <https://arxiv.org/abs/2504.20571>.
- Mingqi Wu, Zhihao Zhang, Qiaole Dong, Zhiheng Xi, Jun Zhao, Senjie Jin, Xiaoran Fan, Yuhao Zhou, Huijie Lv, Ming Zhang, Yanwei Fu, Qin Liu, Songyang Zhang, and Qi Zhang. Reasoning or memorization? unreliable results of reinforcement learning due to data contamination, 2025. URL <https://arxiv.org/abs/2507.10532>.
- Feng Yao, Liyuan Liu, Dinghuai Zhang, Chengyu Dong, Jingbo Shang, and Jianfeng Gao. Your efficient rl framework secretly brings you off-policy rl training, August 2025. URL <https://fengyao.notion.site/off-policy-rl>.
- Yixin Ye, Zhen Huang, Yang Xiao, Ethan Chern, Shijie Xia, and Pengfei Liu. Limo: Less is more for reasoning, 2025. URL <https://arxiv.org/abs/2502.03387>.
- Qingyu Yin, Yulun Wu, Zhennan Shen, Sunbowen Li, Zhilin Wang, Yanshu Li, Chak Tou Leong, Jiale Kang, and Jinjin Gu. Evaluating parameter efficient methods for rlvr, 2025. URL <https://arxiv.org/abs/2512.23165>.
- Elad Ben Zaken, Shauli Ravfogel, and Yoav Goldberg. Bitfit: Simple parameter-efficient fine-tuning for transformer-based masked language-models, 2022. URL <https://arxiv.org/abs/2106.10199>.
- Weihao Zeng, Yuzhen Huang, Qian Liu, Wei Liu, Keqing He, Zejun Ma, and Junxian He. Simplerl-zoo: Investigating and taming zero reinforcement learning for open base models in the wild, 2025. URL <https://arxiv.org/abs/2503.18892>.
- Biao Zhang, Zhongtao Liu, Colin Cherry, and Orhan Firat. When scaling meets llm finetuning: The effect of data, model and finetuning method, 2024a. URL <https://arxiv.org/abs/2402.17193>.
- Qingru Zhang, Minshuo Chen, Alexander Bukharin, Nikos Karampatziakis, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. Adalora: Adaptive budget allocation for parameter-efficient fine-tuning, 2023. URL <https://arxiv.org/abs/2303.10512>.
- Xiaotian Zhang, Chunyang Li, Yi Zong, Zhengyu Ying, Liang He, and Xipeng Qiu. Evaluating the performance of large language models on gaokao benchmark, 2024b. URL <https://arxiv.org/abs/2305.12474>.
- Hongyun Zhou, Xiangyu Lu, Wang Xu, Conghui Zhu, Tiejun Zhao, and Muyun Yang. Lora-drop: Efficient lora parameter pruning based on output evaluation, 2024. URL <https://arxiv.org/abs/2402.07721>.

A Appendix

A.1 Additional ablations

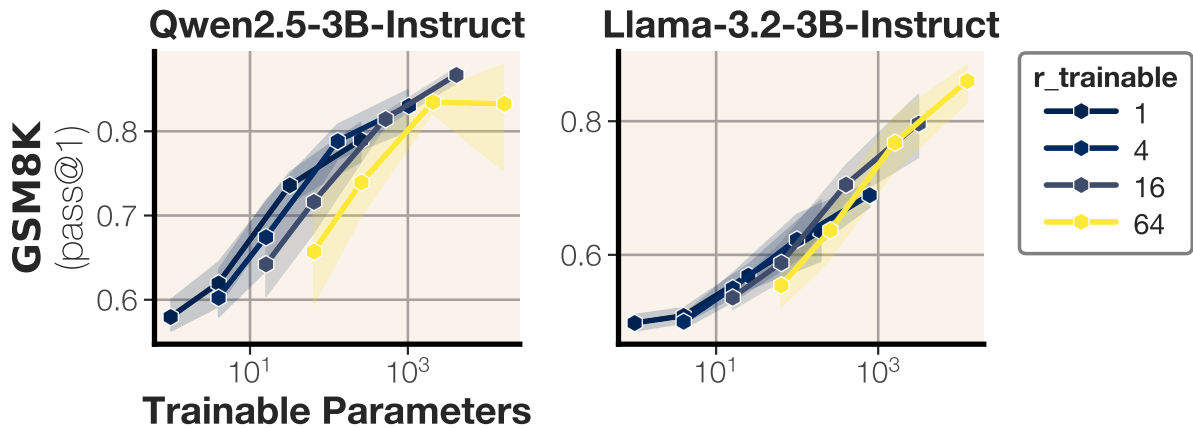


Figure 9 Performance ablation for Qwen2.5-3B-Instruct with different numbers of tied layers and ranks r .

We ran a number of ablations to isolate the right way to ‘spend’ trainable parameters: is it better to have more, shared parameters or fewer-but-unique parameters? We sweep across values of $r_{\text{trainable}}$ for 3B versions of Qwen-2.5 and LLAMA-3.2 on GSM8K. Surprisingly, we find higher performance at matched parameter count for a *lower* value of $r_{\text{trainable}}$, indicating parameters are better spent on fewer, unique parameters. We also observe again that Qwen far outperforms Llama at a constant update size: when training just 100 parameters, Llama achieves 60% accuracy, while Qwen averages 74% across 3 random seeds.